ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 7 - Due date 03/06/26

Joy Wu

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github. And to do so you will need to fork our repository and link it to your RStudio.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A07_Sp26.Rmd"). Then change "Student Name" on line 4 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Sakai.

Packages needed for this assignment: "forecast","tseries". Do not forget to load them before running your script, since they are NOT default packages.\

Set up

```
#Load/install required package here
library(lubridate)
```

```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
library(ggplot2)
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
library(Kendall)
library(tseries)
library(outliers)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr   1.1.4     v stringr 1.5.1
## v forcats 1.0.0     v tibble  3.2.1
## v purrr   1.0.2     v tidyr   1.3.1
## v readr   2.1.5

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(cowplot)
```

```
##
## Attaching package: 'cowplot'
##
## The following object is masked from 'package:lubridate':
##
##     stamp
```

```r
library(dplyr)
library(readr)
```

Importing and processing the data set

Consider the data from the file "Net_generation_United_States_all_sectors_monthly.csv". The data corresponds to the monthly net generation from January 2001 to December 2020 by source and is provided by the US Energy Information and Administration. **You will work with the natural gas column only**.

Q1

Import the csv file and create a time series object for natural gas. Make you sure you specify the **start=** and **frequency=** arguments. Plot the time series over time, ACF and PACF.

```r
net.generation.data <- read.csv(file="./Data/Net_generation_United_States_all_sectors_monthly.csv",head
head(net.generation.data)
```

```
##      Month all.fuels..utility.scale..thousand.megawatthours
## 1 Dec 2020                                          344970.4
## 2 Nov 2020                                          302701.8
## 3 Oct 2020                                          313910.0
## 4 Sep 2020                                          334270.1
## 5 Aug 2020                                          399504.2
## 6 Jul 2020                                          414242.5
```

```
##   coal.thousand.megawatthours natural.gas.thousand.megawatthours
## 1                    78700.33                           125703.7
## 2                    61332.26                           109037.2
## 3                    59894.57                           131658.2
## 4                    68448.00                           141452.7
## 5                    91252.48                           173926.6
## 6                    89831.36                           185444.8
##   nuclear.thousand.megawatthours
## 1                       69870.98
## 2                       61759.98
## 3                       59362.46
## 4                       65727.32
## 5                       68982.19
## 6                       69385.44
##   conventional.hydroelectric.thousand.megawatthours
## 1                                          23086.37
## 2                                          21831.88
## 3                                          18320.72
## 4                                          19161.97
## 5                                          24081.57
## 6                                          27675.94
```

```
nvar <- ncol(net.generation.data) - 1
nobs <- nrow(net.generation.data)

net.generation.data.processed <-
  net.generation.data %>%
  mutate( Month = my(Month) ) %>%
  rename( All.fuels = all.fuels..utility.scale..thousand.megawatthours ) %>%
  rename( Coal = coal.thousand.megawatthours ) %>%
  rename( Natural.gas = natural.gas.thousand.megawatthours ) %>%
  rename( Nuclear = nuclear.thousand.megawatthours ) %>%
  rename( Conventional.hydroelectric = conventional.hydroelectric.thousand.megawatthours) %>%
  arrange( Month )

head(net.generation.data.processed)
```

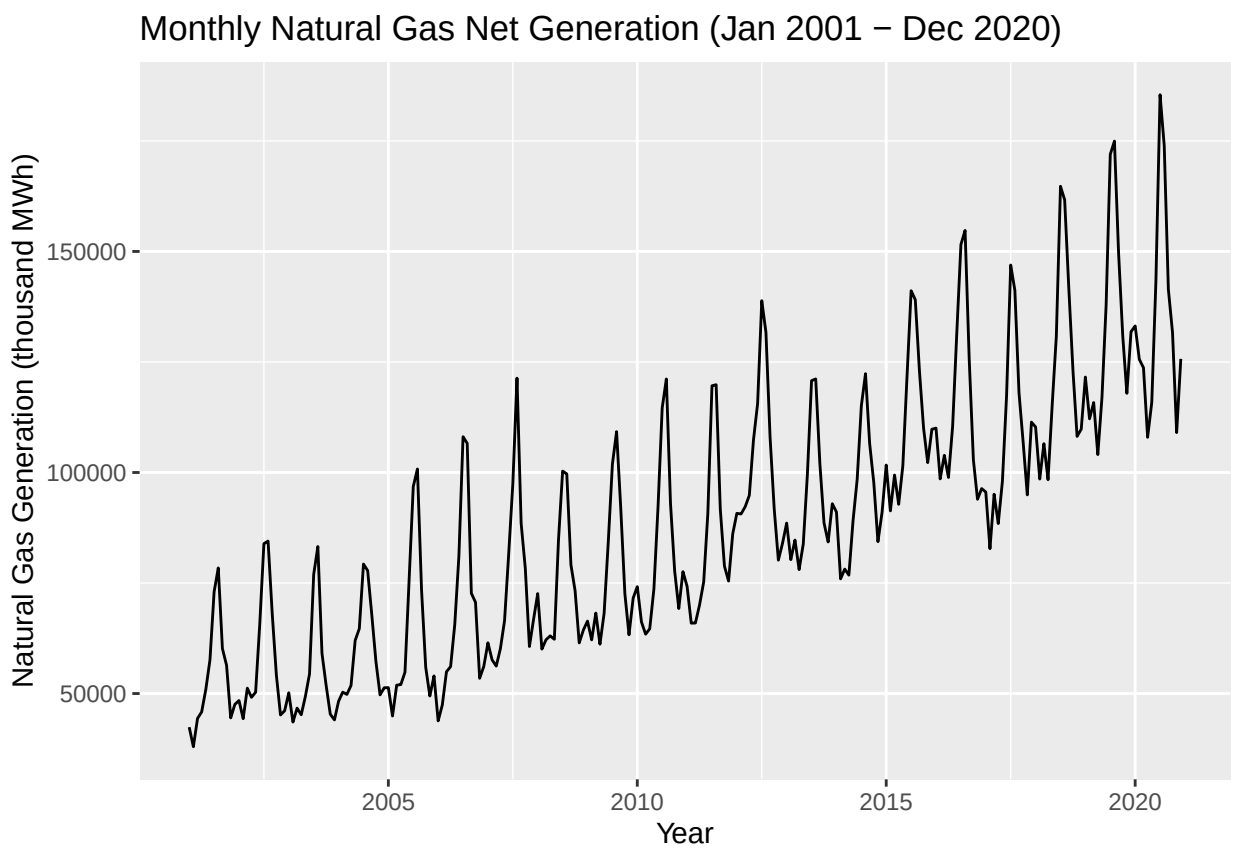
```
##        Month All.fuels     Coal Natural.gas  Nuclear Conventional.hydroelectric
## 1 2001-01-01  332493.2 177287.1    42388.66 68707.08                   18852.05
## 2 2001-02-01  282940.2 149735.5    37966.93 61272.41                   17472.89
## 3 2001-03-01  300706.5 155269.0    44364.41 62140.71                   20477.19
## 4 2001-04-01  278078.9 140670.7    45842.75 56003.03                   18012.99
## 5 2001-05-01  300491.6 151592.9    50934.21 61512.44                   19175.63
## 6 2001-06-01  327694.0 162615.8    57603.15 68023.10                   20727.63
```

```
summary(net.generation.data.processed)
```

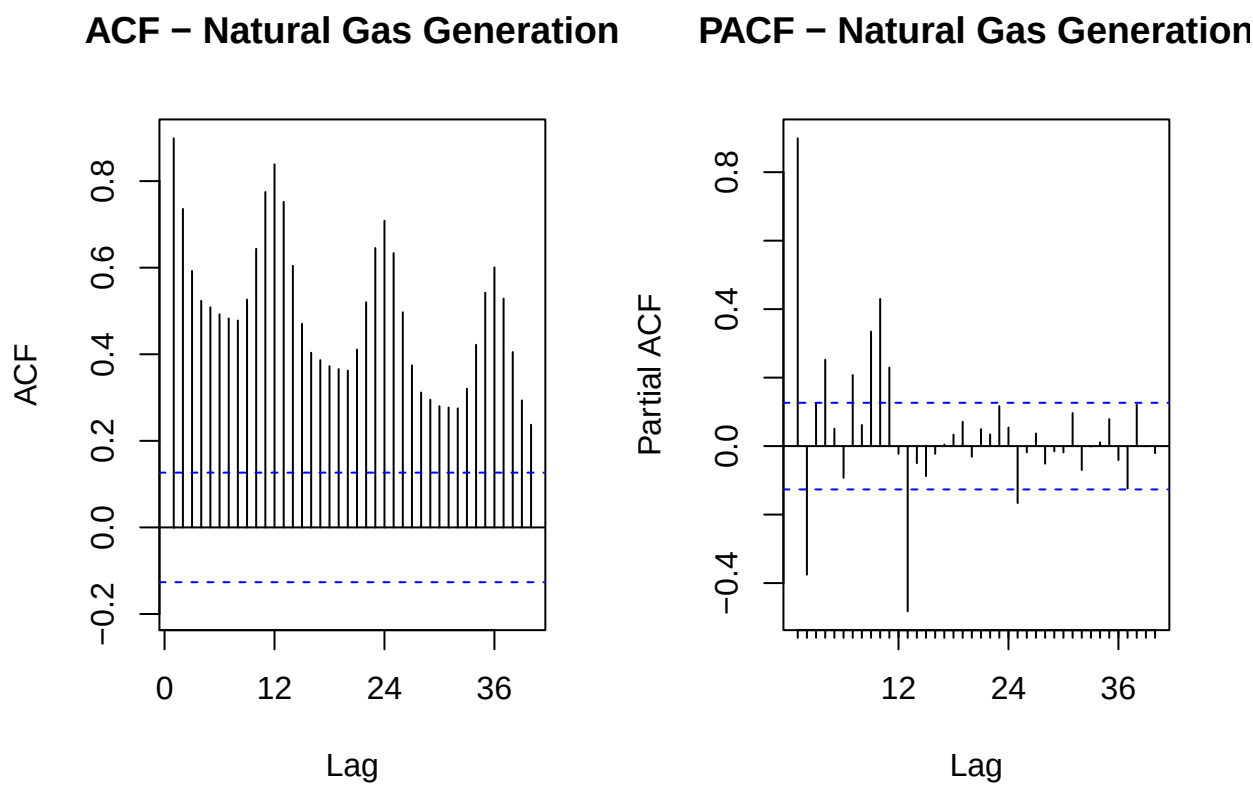
```
##      Month              All.fuels           Coal          Natural.gas
##  Min.   :2001-01-01   Min.   :276127   Min.   : 40624   Min.   : 37967
##  1st Qu.:2005-12-24   1st Qu.:309142   1st Qu.:113769   1st Qu.: 62245
##  Median :2010-12-16   Median :328297   Median :141665   Median : 84415
##  Mean   :2010-12-16   Mean   :336382   Mean   :135391   Mean   : 88028
##  3rd Qu.:2015-12-08   3rd Qu.:357671   3rd Qu.:161751   3rd Qu.:108385
```

```
##  Max.   :2020-12-01   Max.   :421797   Max.   :190135   Max.   :185445
##     Nuclear      Conventional.hydroelectric
##  Min.   :54547   Min.   :14743
##  1st Qu.:62651   1st Qu.:19568
##  Median :65775   Median :22307
##  Mean   :66037   Mean   :22639
##  3rd Qu.:70438   3rd Qu.:25475
##  Max.   :74649   Max.   :32607
```

```
natural.gas.ts <- ts(
  net.generation.data.processed$Natural.gas,
  start = c(year(net.generation.data.processed$Month[1]), month(net.generation.data.processed$Month[1]))
  frequency = 12
)

autoplot(natural.gas.ts) +
  xlab("Year") + ylab("Natural Gas Generation (thousand MWh)") +
  ggtitle("Monthly Natural Gas Net Generation (Jan 2001 - Dec 2020)")
```



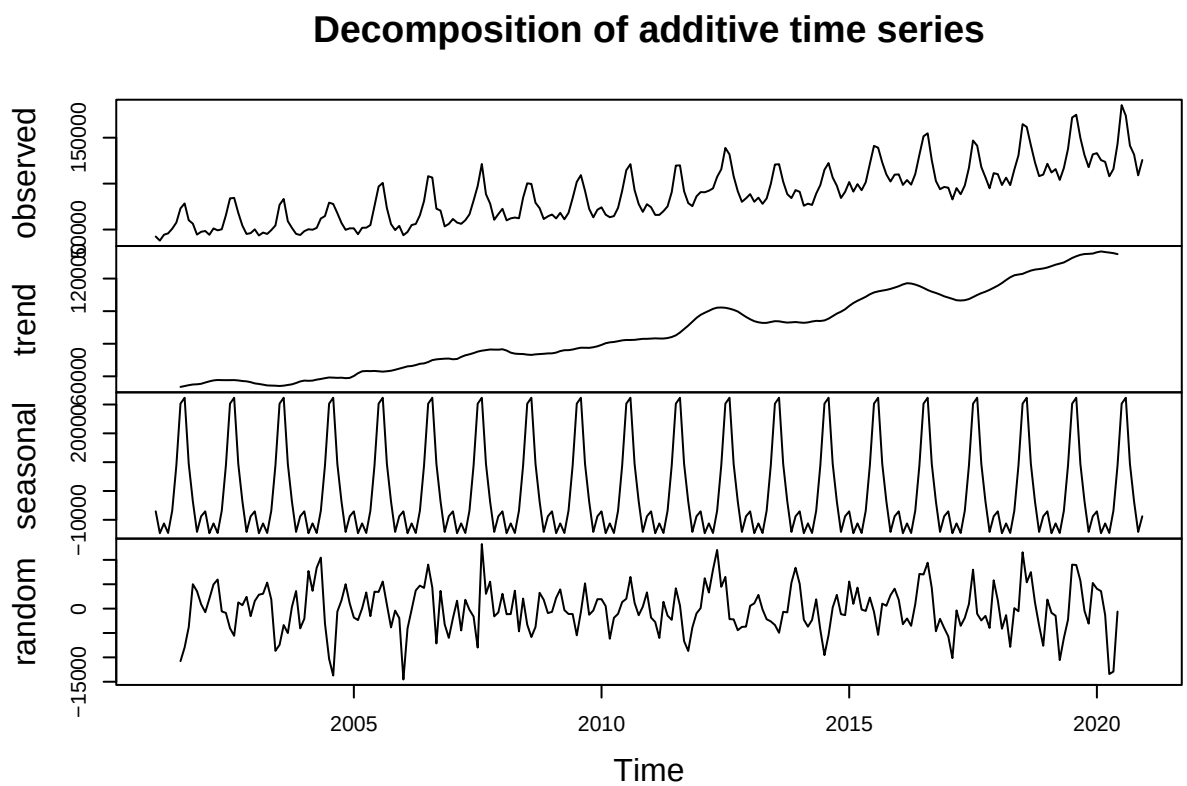

```
par(mfrow = c(1, 2))
natural.gas.acf <- Acf(natural.gas.ts, lag.max = 40, main = "ACF - Natural Gas Generation")
natural.gas.pacf <- Pacf(natural.gas.ts, lag.max = 40, main = "PACF - Natural Gas Generation")
```



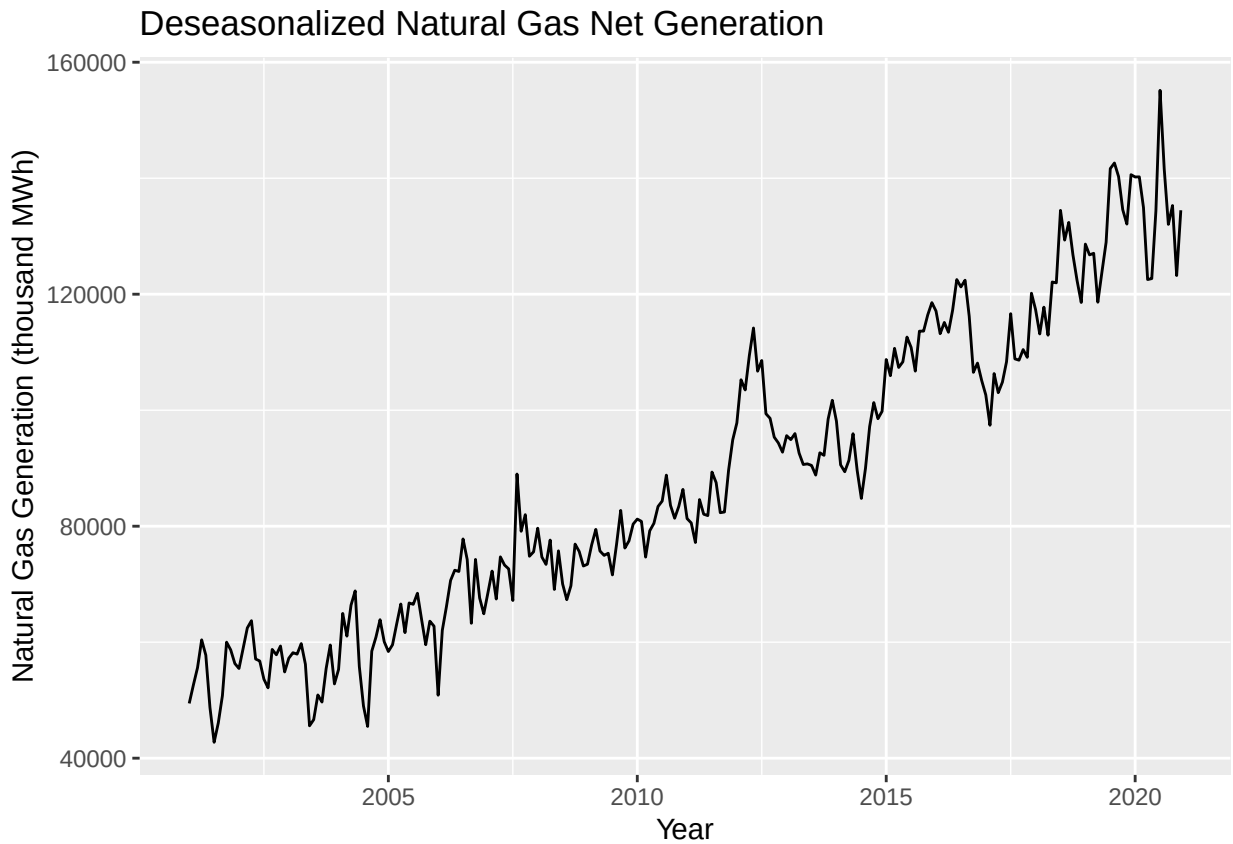
Q2

Using the *decompose*() and the *seasadj*() functions create a series without the seasonal component, i.e., a deseasonalized natural gas series. Plot the deseasonalized series over time and corresponding ACF and PACF. Compare with the plots obtained in Q1.

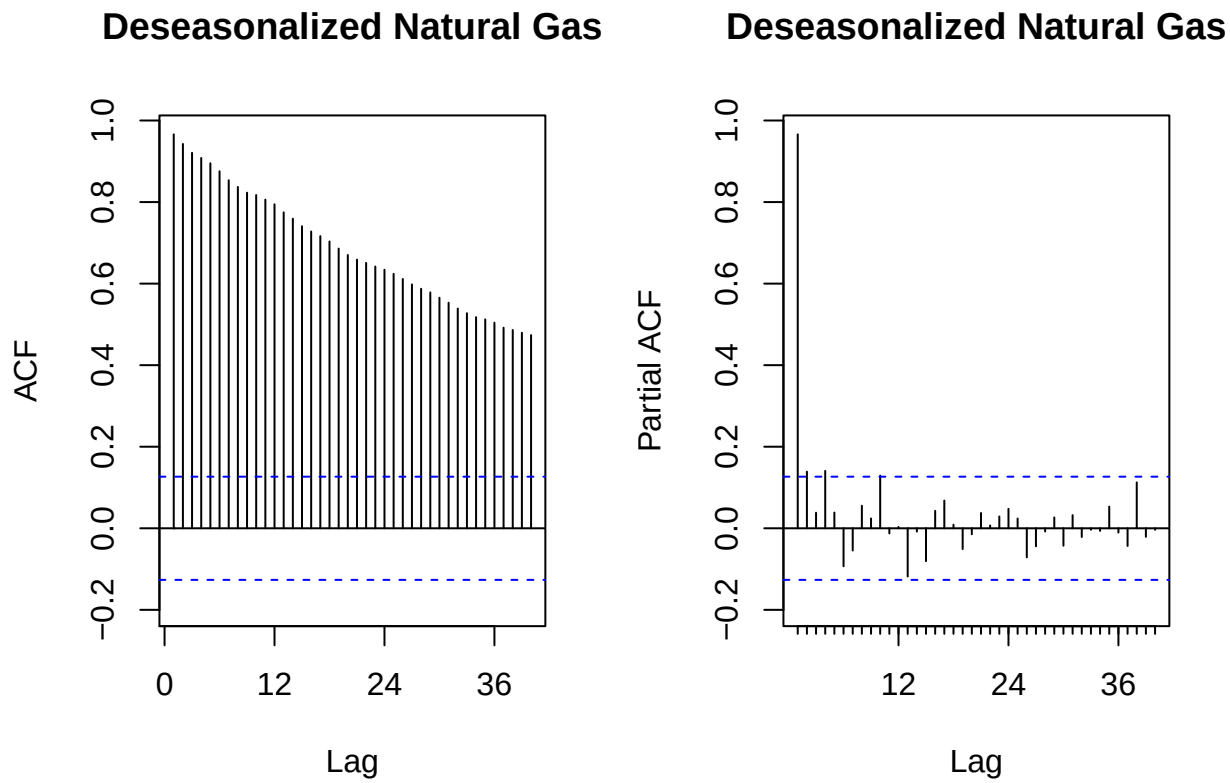
```
decompose.natural.gas <- decompose(natural.gas.ts, "additive")
plot(decompose.natural.gas)
```




```
deseasonal.natural.gas <- seasadj(decompose.natural.gas)

autoplot(deseasonal.natural.gas) +
  xlab("Year") + ylab("Natural Gas Generation (thousand MWh)") +
  ggtitle("Deseasonalized Natural Gas Net Generation")
```




```
par(mfrow = c(1, 2))
deseasonal.acf <- Acf(deseasonal.natural.gas, lag.max = 40, main = "Deseasonalized Natural Gas")
deseasonal.pacf <- Pacf(deseasonal.natural.gas, lag.max = 40, main = "Deseasonalized Natural Gas")
```



>Answer: In Q1, the ACF plot shows strong spikes at seasonal lags (12, 24, 36,...) since seasonality is present. It also shows a decreasing trend since trend is also present. The PACF also show the same pattern (spikes at seasonal lags and slow decay). After deseasonalizing, the ACF plot still shows a slow decay (indicating non-stationarity due to the trend), but the seasonal peaks have been removed. Same for the PACF plot, it shows a dominant spike at lag 1 with a much faster decay, consistent with he trended but non-seasonal process.

Modeling the seasonally adjusted (i.e., the deseasonalized series)

Q3

Run the ADF test and Mann Kendall test on the deseasonalized data from Q2. Report and explain the results.

```
adf.result <- adf.test(deseasonal.natural.gas)
```

```
## Warning in adf.test(deseasonal.natural.gas): p-value smaller than printed
## p-value
```

```
adf.result
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  deseasonal.natural.gas
```

```
## Dickey-Fuller = -4.0271, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```
mk.result <- MannKendall(deseasonal.natural.gas)
mk.result
```

```
## tau = 0.843, 2-sided pvalue =< 2.22e-16
```

> Answer: The ADF test shows that the p value is 0.01. Since it tests whether the series has a unit root (non-stationary), having a p value this small (<0.05) means that we would reject the null hypothesis. This indicates that the series is stationary. This looks like it contradicts the above plots, which shows non-stationarity. This would suggest that although the series exhibits strong autocorrelation, it is statistically stationary. The MK results show that the p value is 2.22e-16 (< 0.05), indicating that there is significant deterministic trend present in the series.

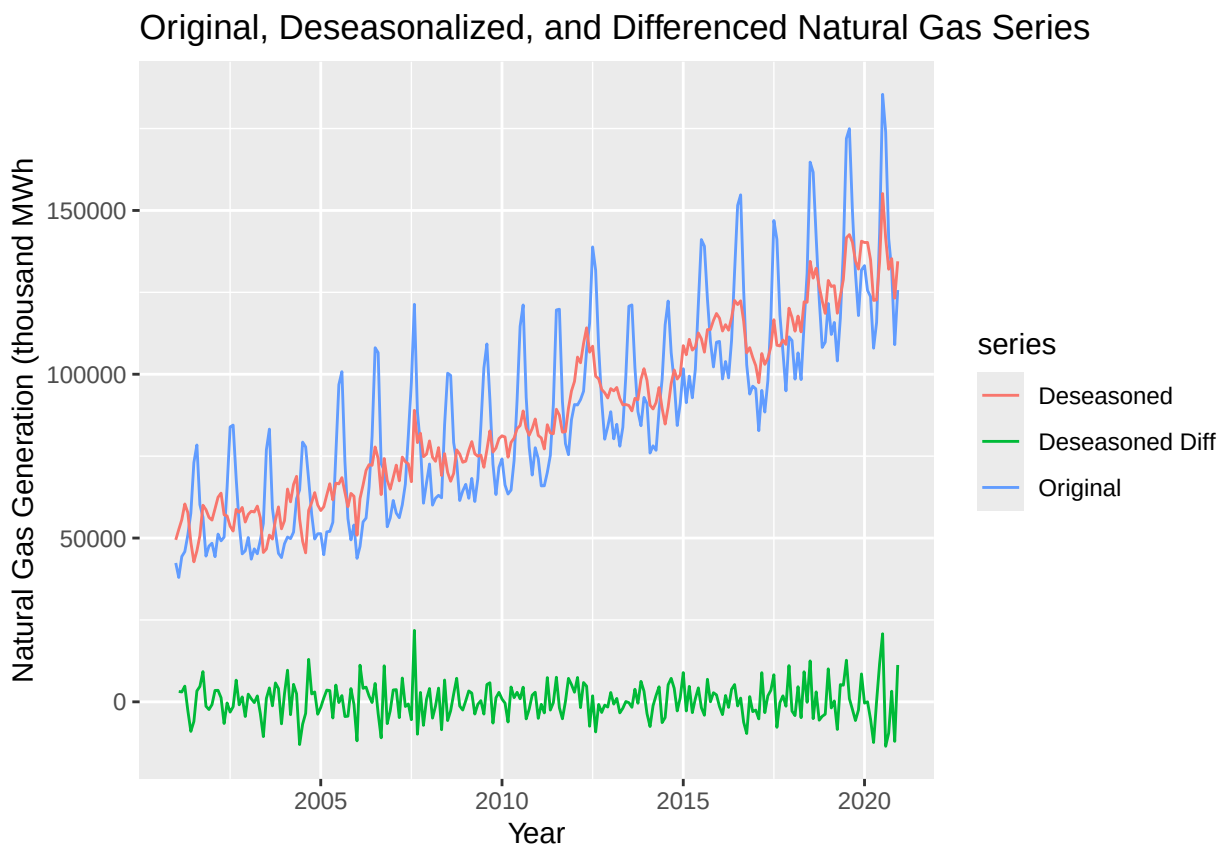
Q4

Using the plots from Q2 and test results from Q3 identify the ARIMA model parameters p, d and q. Note that in this case because you removed the seasonal component prior to identifying the model you don't need to worry about seasonal component. Clearly state your criteria and any additional function in R you might use. DO NOT use the *auto.arima*() function. You will be evaluated on ability to understand the ACF/PACF plots and interpret the test results.

```
n_diff <- ndiffs(deseasonal.natural.gas)
cat("Number of differencing needed: ",n_diff)
```

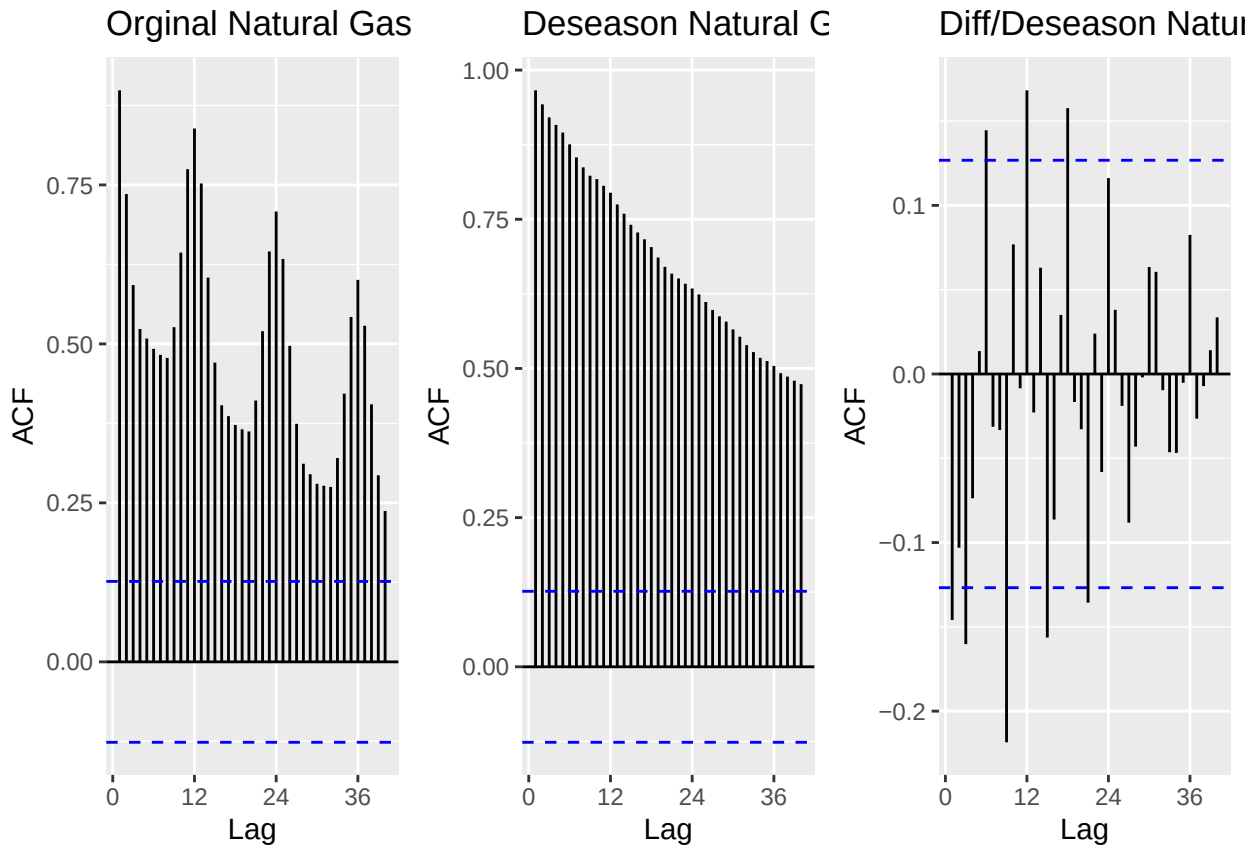
```
## Number of differencing needed:  1
```

```
deseasonal.natural.gas.diff <- diff(deseasonal.natural.gas, differences = 1, lag = 1)

autoplot(natural.gas.ts, series="Original") +
  autolayer(deseasonal.natural.gas, series="Deseasoned") +
  autolayer(deseasonal.natural.gas.diff, series="Deseasoned Diff") +
  xlab("Year") + ylab("Natural Gas Generation (thousand MWh") +
  ggtitle("Original, Deseasonalized, and Differenced Natural Gas Series")
```



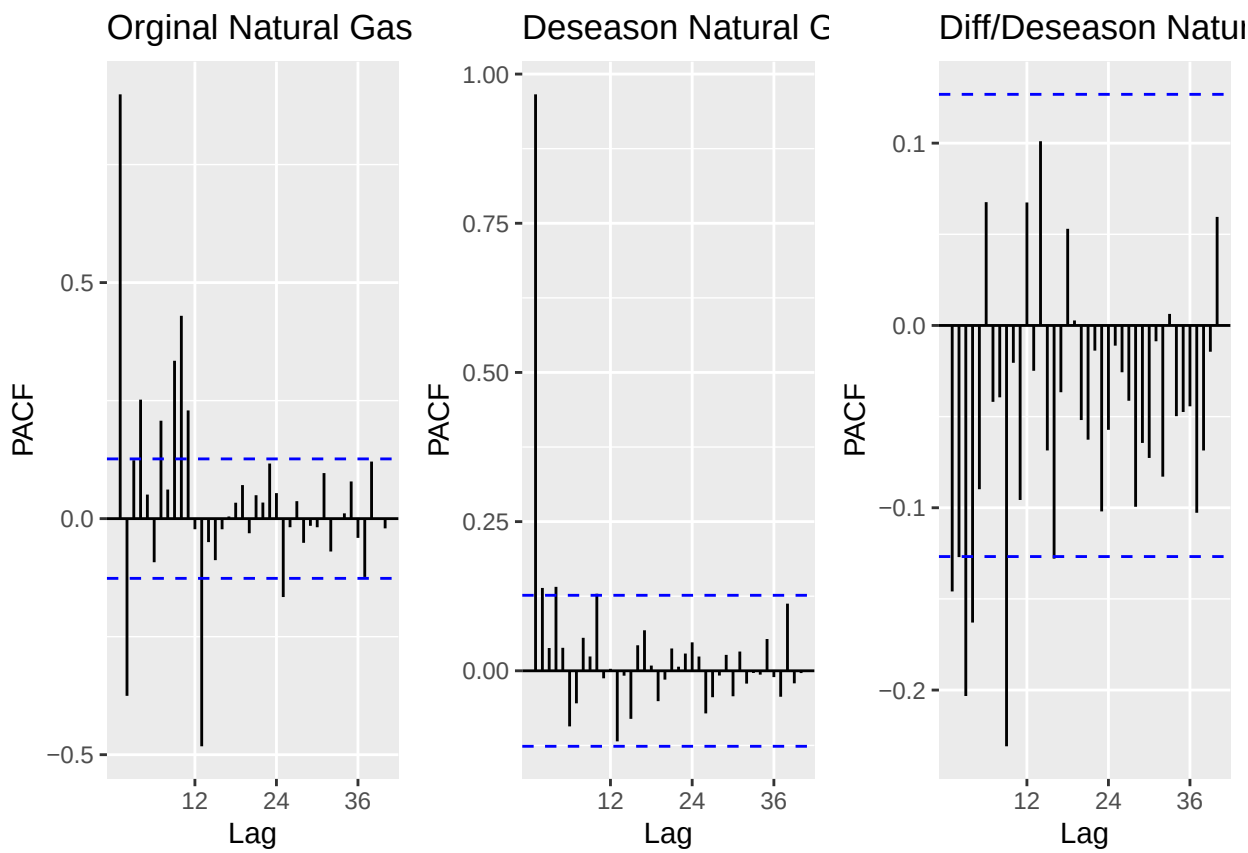

```r
plot_grid(
  autoplot(Acf(natural.gas.ts, lag = 40, plot=FALSE),
                 main = "Orginal Natural Gas Series"),
  autoplot(Acf(deseasonal.natural.gas, lag = 40, plot=FALSE),
                   main = "Deseason Natural Gas Series"),
  autoplot(Acf(deseasonal.natural.gas.diff, lag = 40, plot=FALSE),
                   main = "Diff/Deseason Natural Gas Series"),
  nrow=1
)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
```



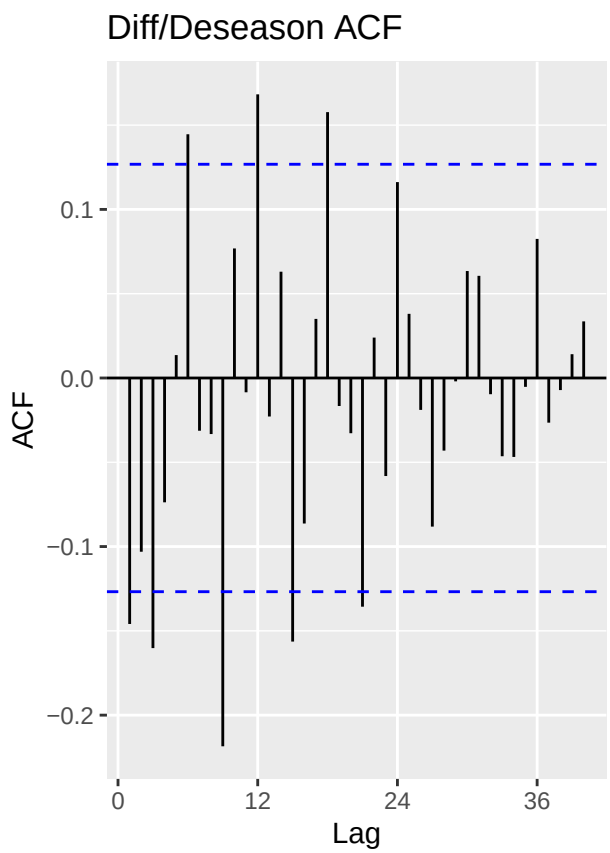

```r
plot_grid(
  autoplot(Pacf(natural.gas.ts, lag = 40, plot=FALSE),
                 main = "Orginal Natural Gas Series"),
  autoplot(Pacf(deseasonal.natural.gas, lag = 40, plot=FALSE),
                   main = "Deseason Natural Gas Series"),
  autoplot(Pacf(deseasonal.natural.gas.diff, lag = 40, plot=FALSE),
                   main = "Diff/Deseason Natural Gas Series"),
  nrow=1
)
```

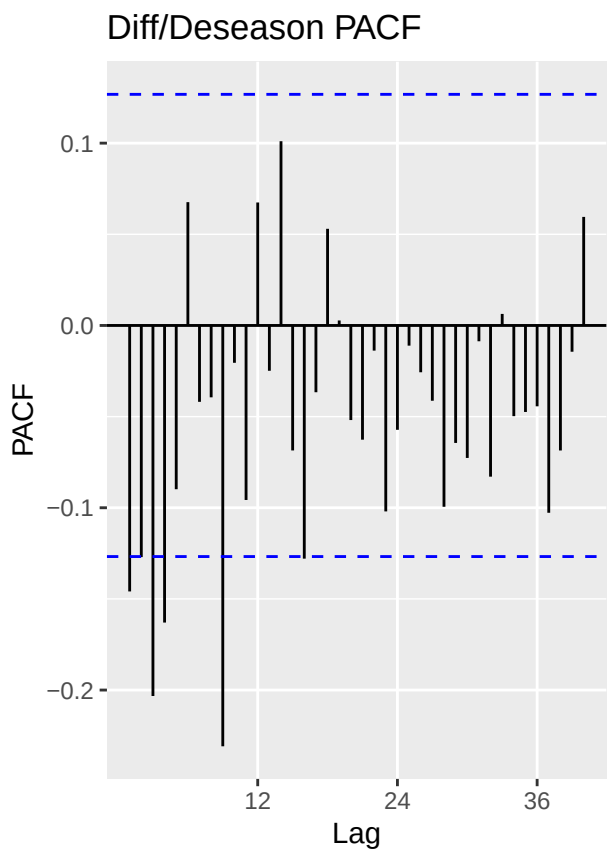
```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
```




```r
plot_grid(
  autoplot(Acf(deseasonal.natural.gas.diff, lag.max = 40, plot = FALSE),
           main = "Diff/Deseason ACF"),
  autoplot(Pacf(deseasonal.natural.gas.diff, lag.max = 40, plot = FALSE),
           main = "Diff/Deseason PACF"),
  nrow = 1
)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
```





> Answer: By using the ndiffs() function, we can determine that the differencing order is 1. Based on the ACF, we can determine that the order q = 1 since the first spike is significant, and the values following that quickly fall within the blue dashed lines. This suggests that it's a MA process with an order of 1. Based on the PACF plot, we can determine that the order is also 1 (p = 1) here since it shows a spike at lag 1, with the following values falling within the blue dashed line. This suggests that it's an AR(1) process. However, since both plots are not showing a clear textbook-clean pattern, we know that it's an ARIMA model with both process components present. Therefore, the identified model is ARIMA(1,1,1). Additional R functions that could be used to help determine the model is Arima(), which lets us manually fit the values and try to determine the model.

Q5

Use `Arima()` from package "forecast" to fit an ARIMA model to your series considering the order estimated in Q4. You should allow constants in the model, i.e., `include.mean = TRUE` or `include.drift=TRUE`. **Print the coefficients** in your report. Hint: use the `cat()` or `print()` function to print.

```
arima.fit <- Arima(deseasonal.natural.gas,
                   order = c(1,1,1),
                   include.drift = TRUE)
print(arima.fit)
```

```
## Series: deseasonal.natural.gas
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
```

```
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
## AIC=4774.21   AICc=4774.38   BIC=4788.12
```

```r
cat("\nEstimated coefficients ARIMA(1,1,1):\n")
```

```
##
## Estimated coefficients ARIMA(1,1,1):
```

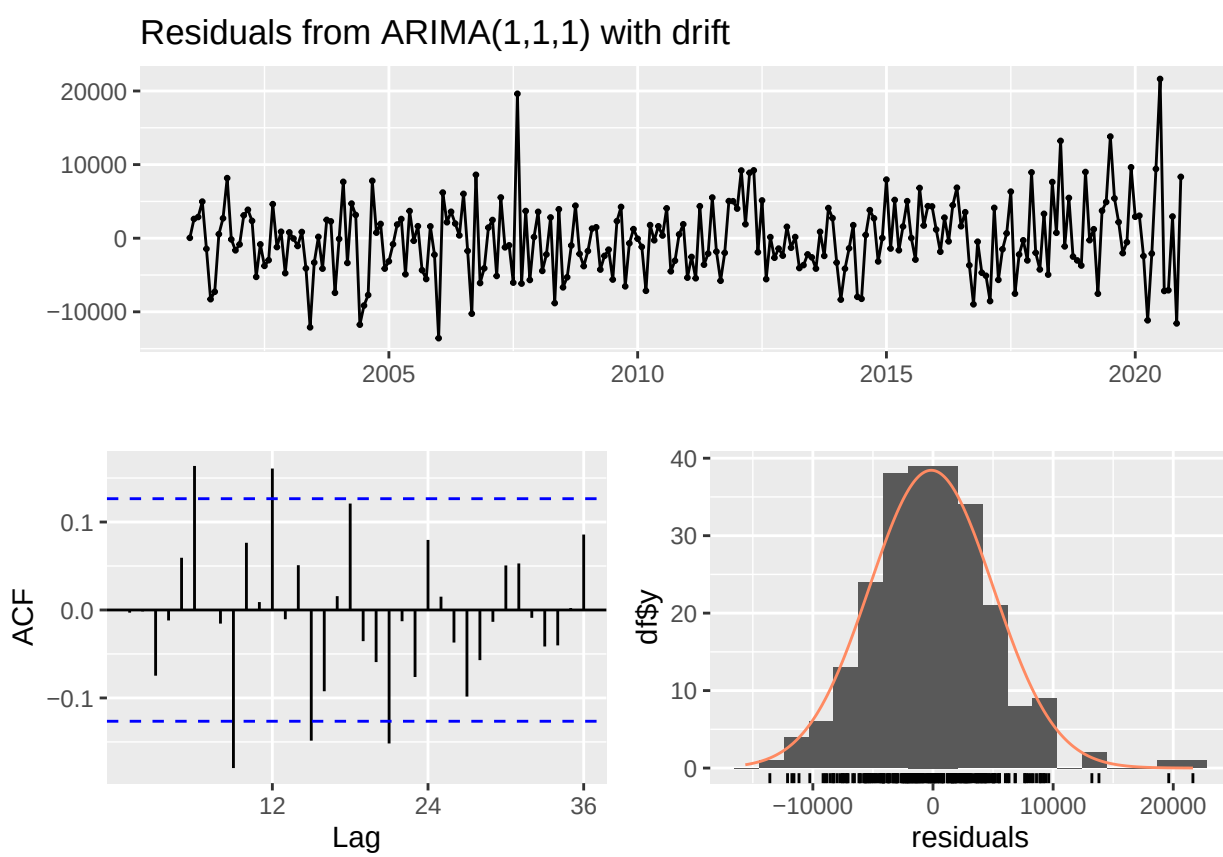
```r
print(coef(arima.fit))
```

```
##         ar1         ma1       drift
##   0.7065237  -0.9794655 359.5051902
```

Q6

Now plot the residuals of the ARIMA fit from Q5 along with residuals ACF on the same window. You may use the *checkresiduals*() function to get the plots. Do the residual series look like a white noise series? Why?

```r
checkresiduals(arima.fit)
```



```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1) with drift
## Q* = 48.356, df = 22, p-value = 0.0009736
##
## Model df: 2.   Total lags used: 24
```

> Answer: The residual series looks like a white noise series. The residual time series plot at the top shows the residuals fluctuate randomly around zero throughout the whole series with one spike around 2007-2008 and 2021. But the overall pattern suggest that it generally fits the white noise series pattern. In addition, the ACF plot shows no significant spikes, and the histogram shows that the residuals are appromixately normally distributed and centered near zero. These all fit the white noise series pattern.

Modeling the original series (with seasonality)

Q7

Repeat Q3-Q6 for the original series (the complete series that has the seasonal component). Note that when you model the seasonal series, you need to specify the seasonal part of the ARIMA model as well, i.e., P, D and Q.

Q3 (original series)

```r
adf.result.original <- adf.test(natural.gas.ts)
```

```
## Warning in adf.test(natural.gas.ts): p-value smaller than printed p-value
```

```r
adf.result.original
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  natural.gas.ts
## Dickey-Fuller = -8.9602, Lag order = 6, p-value = 0.01
## alternative hypothesis: stationary
```

```r
mk.result.original <- MannKendall(natural.gas.ts)
mk.result.original
```

```
## tau = 0.651, 2-sided pvalue =< 2.22e-16
```

> Answer: Both ADF and MK test show the same results as when we tested them for the deseasoned series. The ADF test shows that the p value is 0.01. Since it tests whether the series has a unit root (non-stationary), having a p value this small (<0.05) means that we would reject the null hypothesis. This indicates that the series is stationary. The MK results show that the p value is 2.22e-16 (< 0.05), indicating that there is significant deterministic trend present in the series.

Q4 (original series)

```r
n_diff.original <- ndiffs(natural.gas.ts)
cat("Number of differencing needed: ",n_diff.original)
```

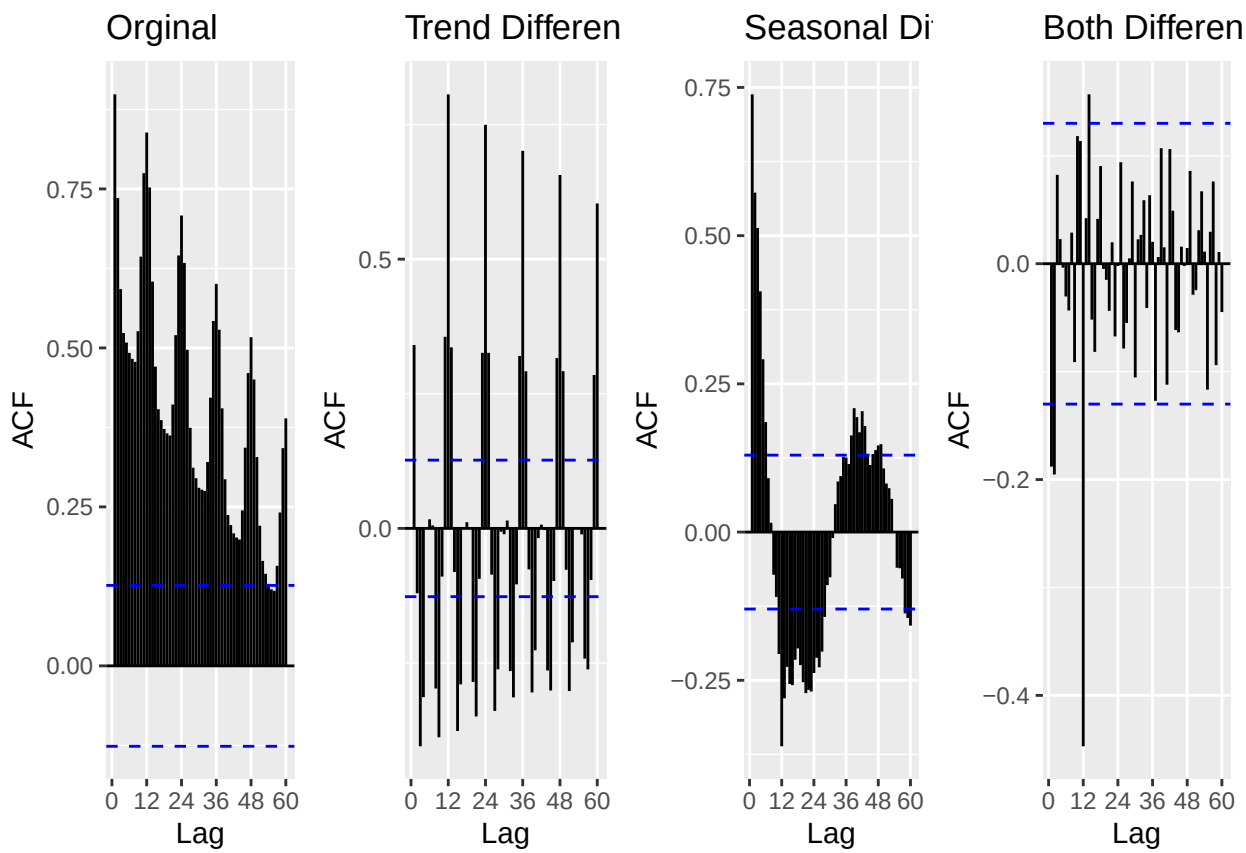
```
## Number of differencing needed:  1
```

```r
ns_diff.original <- nsdiffs(natural.gas.ts)
cat("Number of differencing needed: ",ns_diff.original)
```

```
## Number of differencing needed:  1
```

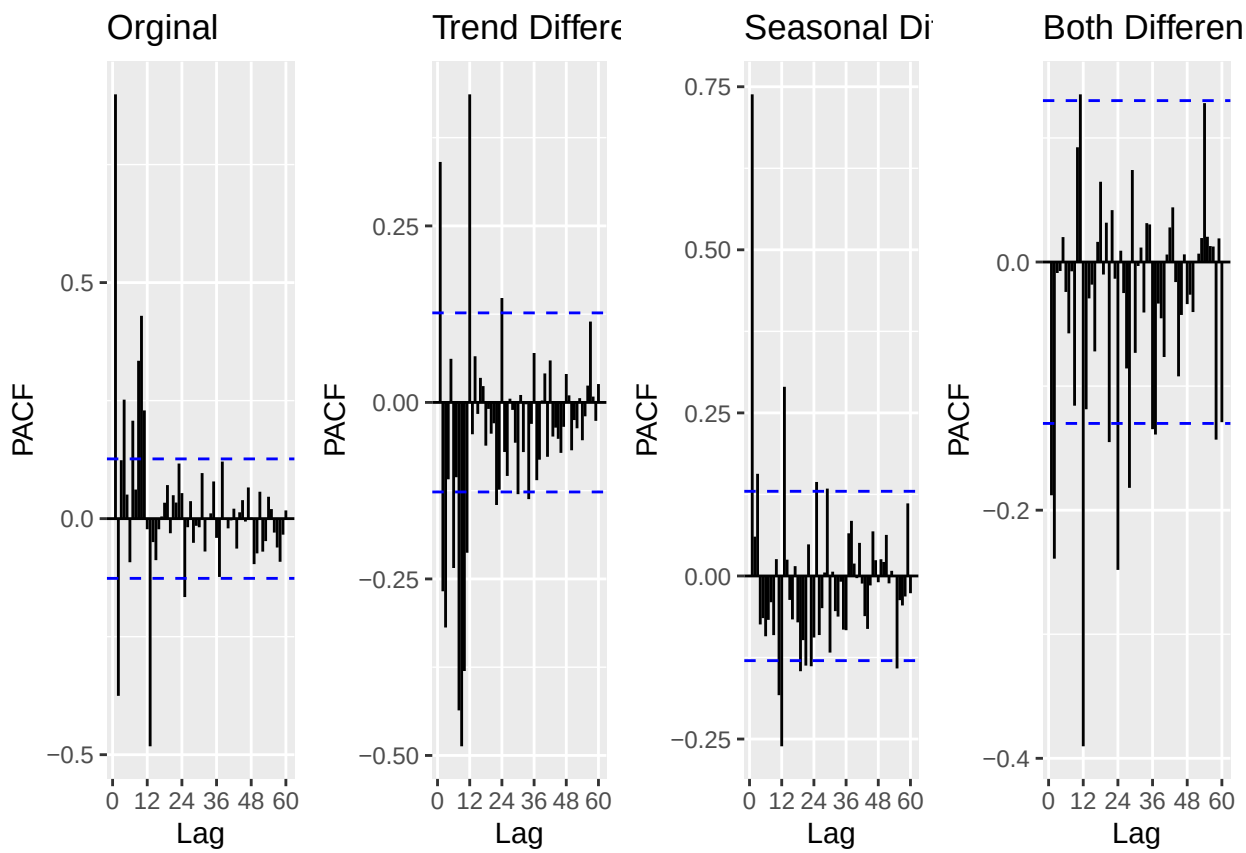
```r
natural.gas.trend.diff <- diff(natural.gas.ts, lag = 1, differences = 1)
natural.gas.seas.diff <- diff(natural.gas.ts, lag = 12, differences = 1)
natural.gas.both.diff <- diff(natural.gas.trend.diff, lag = 12, differences = 1)

plot_grid(
  autoplot(Acf(natural.gas.ts, lag.max = 60, plot=FALSE),
               main = "Orginal"),
  autoplot(Acf(natural.gas.trend.diff, lag.max = 60, plot=FALSE),
                 main = "Trend Differenced"),
  autoplot(Acf(natural.gas.seas.diff, lag.max = 60, plot=FALSE),
                 main = "Seasonal Differenced"),
  autoplot(Acf(natural.gas.both.diff, lag.max = 60, plot=FALSE),
                 main = "Both Differenced"),
  nrow=1
)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
```



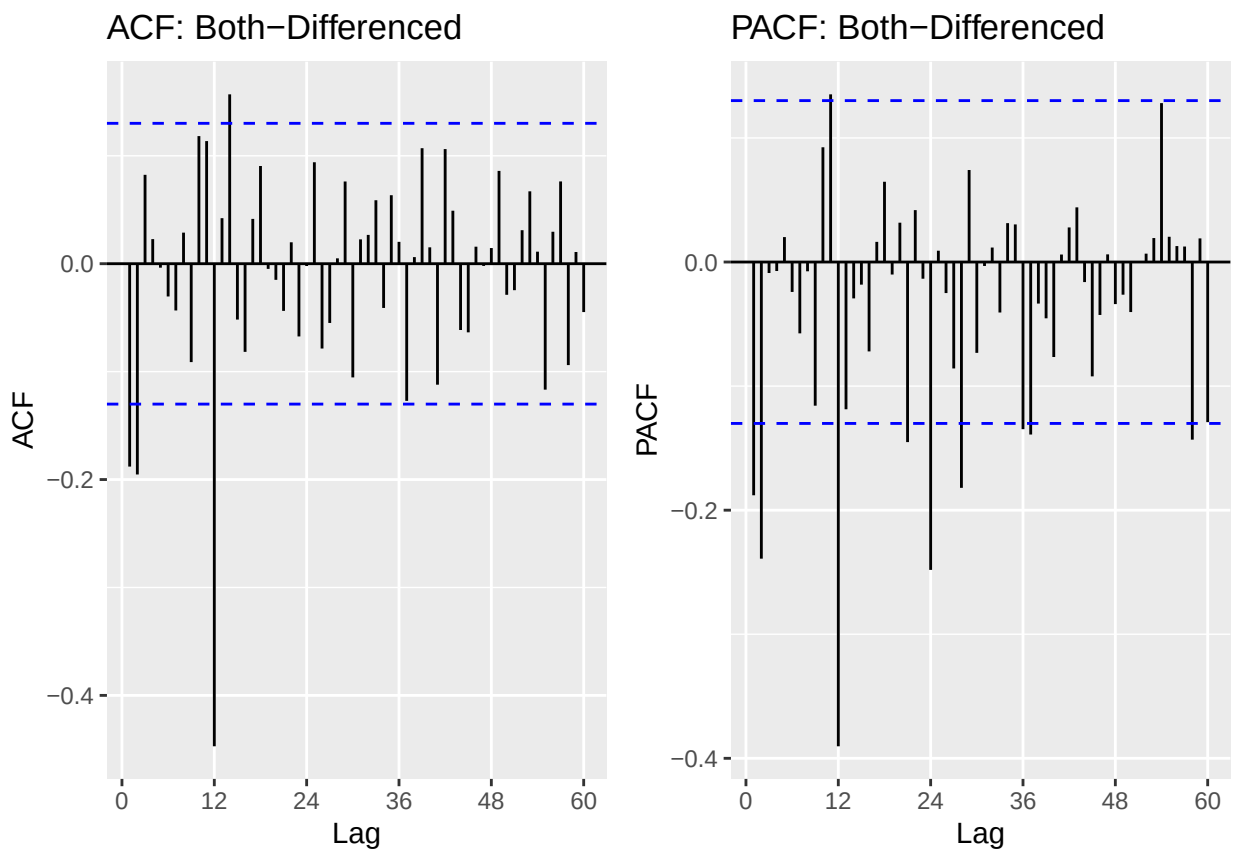

```
plot_grid(
  autoplot(Pacf(natural.gas.ts, lag.max = 60, plot=FALSE),
                main = "Orginal"),
  autoplot(Pacf(natural.gas.trend.diff, lag.max = 60, plot=FALSE),
                  main = "Trend Differenced"),
  autoplot(Pacf(natural.gas.seas.diff, lag.max = 60, plot=FALSE),
                  main = "Seasonal Differenced"),
  autoplot(Pacf(natural.gas.both.diff, lag.max = 60, plot=FALSE),
                  main = "Both Differenced"),
  nrow=1
)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
```




```
plot_grid(
  autoplot(Acf(natural.gas.both.diff, lag.max = 60, plot = FALSE),
           main = "ACF: Both-Differenced"),
  autoplot(Pacf(natural.gas.both.diff, lag.max = 60, plot = FALSE),
           main = "PACF: Both-Differenced"),
  nrow = 1
)
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown parameters: `main`
## Ignoring unknown parameters: `main`
```



>Answer: By using the ndiffs() function, we can determine that the non-seasonal differencing order is 1 (d = 1). By using the nsdiffs() function, we can determine that the seasonal differencing order is also 1 (D = 1), meaning that both a trend difference and a seasonal difference at lag 12 are needed for the original series. After applying both differences, we can look at the both-differenced ACF and PACF plots to identify the remaining model orders. For the non-seasonal component, the ACF shows a spike at lag 2 that quickly falls within the blue dashed area after that, suggesting a MA(2) process with q = 2. The PACF also shows a spike at lag 2 with remaining lags falling within the dashed area, suggesting an AR(2) process with p = 2. Since neither plot shows a perfectly clean cutoff, we can conclude a mixed ARMA component is present, giving p = 2 and q = 2. For the For the seasonal component, both plots show a large negative spike at lag 12 that cuts off in the ACF at lags 24, 36, and 48, suggesting a seasonal MA(1) process. This gives us Q = 1. The PACF spike at lag 12 is similarly large but tails off afterwards, thus giving us P = 0. Overall, this suggests the identified model of SARIMA(1,1,1)(0,1,1)12.

Q5(original series)

```
sarima.fit <- Arima(natural.gas.ts,
                    order = c(1,1,1),
                    seasonal = list(order = c(0,1,1)),
                    include.drift = FALSE)
print(sarima.fit)
```

```
## Series: natural.gas.ts
## ARIMA(1,1,1)(0,1,1)[12]
##
## Coefficients:
##          ar1      ma1     sma1
##       0.7323  -0.9819  -0.7017
```

```
## s.e.  0.0504   0.0183   0.0563
##
## sigma^2 = 27922085:  log likelihood = -2272.2
## AIC=4552.39   AICc=4552.57   BIC=4566.09
```

```r
cat("\nEstimated coefficients SARIMA(1,1,1)(0,1,1)[12]:\n")
```

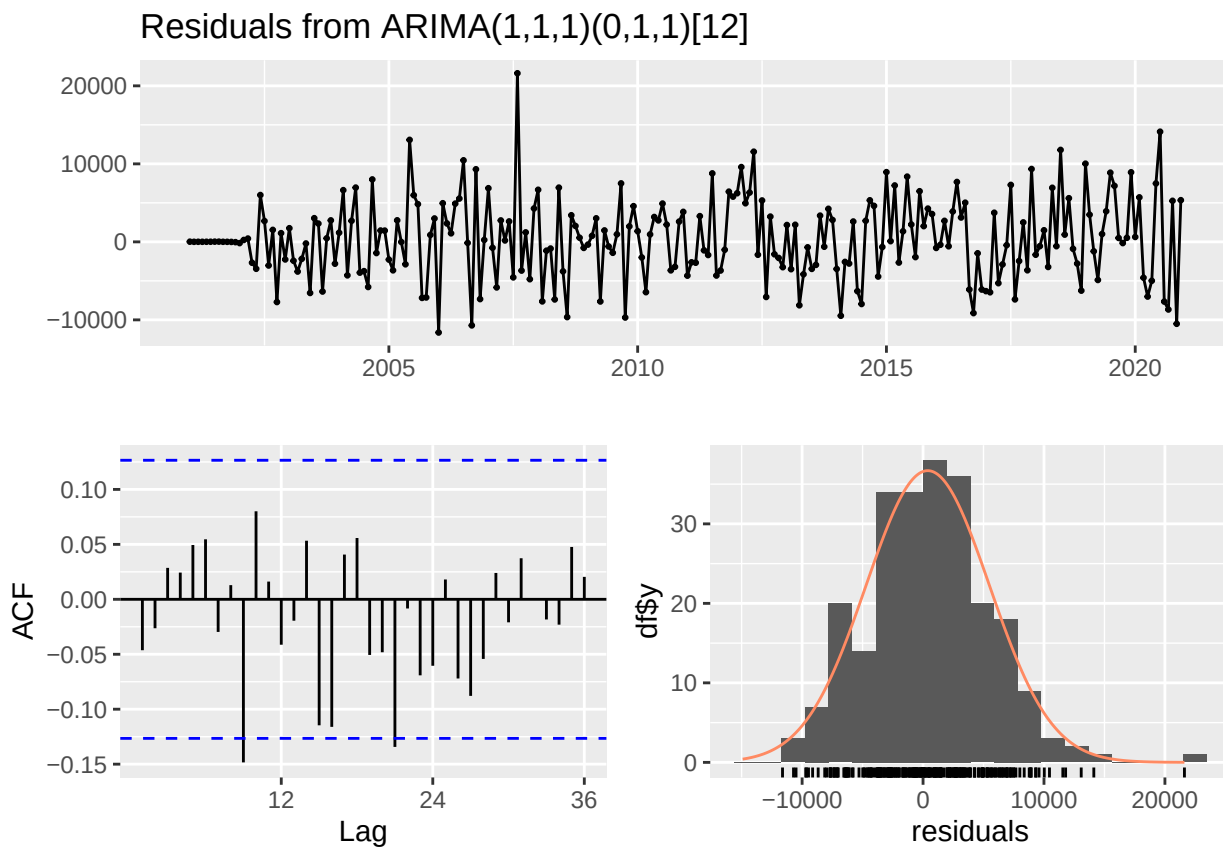
```
##
## Estimated coefficients SARIMA(1,1,1)(0,1,1)[12]:
```

```r
print(coef(sarima.fit))
```

```
##        ar1        ma1       sma1
##  0.7323143 -0.9819269 -0.7016664
```

Q6 (original series)

```r
checkresiduals(sarima.fit)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(1,1,1)(0,1,1)[12]
## Q* = 27.607, df = 21, p-value = 0.1516
##
## Model df: 3.   Total lags used: 24
```

Answer: The residual plots from the SARIMA model is consistent with a white noise series. Similar to the previous one, the residual time series plot at the top shows the residuals fluctuate randomly around zero throughout the whole series with one spike around 2007-2008 and 2021. But the overall pattern suggest that it generally fits the white noise series pattern. In addition, the ACF plot shows no significant spikes at seasonal lags, and the histogram shows that the residuals are appromixately normally distributed and centered near zero. These all fit the white noise series pattern. Compared to the above plots, the SARIMA plot is more complete since it models the original series.

Q8

Compare the residual series for Q7 and Q6. Can you tell which ARIMA model is better representing the Natural Gas Series? Is that a fair comparison? Explain your response.

```
cat("ARIMA(1,1,1) AIC: ", arima.fit$aic, "\n")
```

```
## ARIMA(1,1,1) AIC:  4774.213
```

```
cat("SARIMA(1,1,1)(0,1,1) AIC: ", sarima.fit$aic, "\n")
```

```
## SARIMA(1,1,1)(0,1,1) AIC:  4552.393
```

Answer: Comparing both residual series plots, it's hard to identify which is a better representation since both plots show that they fit the white noise series. Therefore, we would need to look at the AIC. From the result, we can see that the ARIMA model has an AIC of 4774.2, and the SARIMA model has an AIC of 4552. Generally, a lower AIC indicates a better fitting model. Thus, the SARIMA model is a better fit. However, this is not fair comparison since the two models are are modeling different dependent variables. The ARIMA model was fitted on the already deseasonalized series, whereas the SARIMA model was fitted on the original series with seasonality included. When the two models used different series variables but we still fitted them with the same ARIMA framework, the residuals from that would not be directly comparable.

Checking your model with the auto.arima()

Please do not change your answers for Q4 and Q7 after you ran the *auto.arima*(). It is **ok** if you didn't get all orders correctly. You will not loose points for not having the same order as the *auto.arima*().

Q9

Use the *auto.arima*() command on the **deseasonalized series** to let R choose the model parameter for you. What's the order of the best ARIMA model? Does it match what you specified in Q4?

```
auto.arima.fit <- auto.arima(deseasonal.natural.gas, max.D=0,max.P = 0,max.Q=0)
print(auto.arima.fit)
```

```
## Series: deseasonal.natural.gas
## ARIMA(1,1,1) with drift
##
## Coefficients:
##          ar1      ma1     drift
```

```
##       0.7065  -0.9795  359.5052
## s.e.  0.0633   0.0326   29.5277
##
## sigma^2 = 26980609:  log likelihood = -2383.11
## AIC=4774.21   AICc=4774.38   BIC=4788.12
```

Answer: The order identified for the best ARIMA model is ARIMA(1,1,1). This matches with what I identified, and the coefficients and AIC also matched with what I identified earlier in Q4.

Q10

Use the *auto.arima*() command on the **original series** to let R choose the model parameters for you. Does it match what you specified in Q7?

```r
auto.sarima.fit <- auto.arima(natural.gas.ts)
print(auto.sarima.fit)
```

```
## Series: natural.gas.ts
## ARIMA(1,0,0)(0,1,1)[12] with drift
##
## Coefficients:
##          ar1     sma1     drift
##       0.7416  -0.7026  358.7988
## s.e.  0.0442   0.0557   37.5875
##
## sigma^2 = 27569124:  log likelihood = -2279.54
## AIC=4567.08   AICc=4567.26   BIC=4580.8
```

Answer: The best model that auto arima identified is ARIMA(1,0,0)(0,1,1)[12]. The seasoonal component matches with what I have identified, but the non-seasonal component does not. The best model identified D = 0 and d = 1. My interpretation of this is that since D = 1 is applied, it already removes much of the seasonal trend in the original series. This would reduce the need for non-seasonal differencing, thus making the auto process choose d = 1. With this information in mind, I looked back at the earlier question (I didn't change my answer), and realized that instead of looking at the twice differenced plots, I should be focusing on the seasonal differenced only plots because applying both differences when d = 0 is over differencing. From the seasonal differenced only plots, ACF tails off while PACF has a large spike at lag 1 then cuts off afterwards. This suggests an order of p = 1 and q = 0, which then matches the best model that auto arima identified for the non-seasonal part.